

State of the Art — PR-to-Requirements

Automatic Requirement Extraction from Pull Requests using LLM-based Agents

Bozza 0.1 — materiale per repository e successiva integrazione nella tesi

1. Contesto e obiettivo

Le pull request (PR) sono nate come meccanismo di integrazione e revisione delle modifiche al codice, ma nel tempo sono diventate anche artefatti socio-tecnici che raccolgono una parte rilevante della conoscenza prodotta durante l'evoluzione di un sistema software. Titolo, descrizione, discussioni, review, commit e collegamenti ad altri artefatti possono contenere informazioni sul problema affrontato, sul comportamento atteso del sistema, sulle motivazioni della modifica e sulle decisioni progettuali adottate.

Il progetto **PR-to-Requirements** si colloca in questo contesto e studia la possibilità di ricostruire automaticamente requisiti software funzionali a partire da una pull request mediante Large Language Models (LLM) e agenti specializzati. L'architettura prevista comprende un **Requirement Generation Agent**, incaricato di produrre un requisito candidato, e un **Requirement Assessment Agent**, incaricato di valutarne qualità, chiarezza, fedeltà all'evidenza e conformità allo stile richiesto. Se il requisito non soddisfa i criteri stabiliti, il feedback del valutatore viene utilizzato per una nuova generazione. I requisiti approvati vengono inoltre conservati in una memoria persistente accessibile tramite **Model Context Protocol (MCP)**, così da supportare controlli rispetto ai requisiti già ricostruiti, in particolare duplicazioni, sovrapposizioni e possibili incoerenze.

Nel perimetro attuale dello stage, l'input primario del sistema è costituito da **titolo e corpo della pull request**. La letteratura mostra tuttavia che informazioni rilevanti possono essere distribuite anche in commenti, review, issue collegate, commit e diff; tali artefatti rappresentano quindi un'estensione naturale del problema e, se disponibili nel dataset, possono essere utili almeno per la costruzione del gold standard e per l'analisi degli errori.

È inoltre utile distinguere tra *requirement extraction* e *requirement reconstruction*. Il sistema non si limita necessariamente a estrarre una frase già presente nella PR: deve sintetizzare in forma di requisito funzionale un'intenzione che può essere espressa in modo incompleto o indiretto. Per questo motivo, nel seguito si usa prevalentemente il termine **ricostruzione del requisito**, pur mantenendo “automatic requirement extraction” come denominazione generale del task.

2. Pull request come fonte di conoscenza sui requisiti

2.1 Le PR non contengono soltanto codice

Uno dei presupposti fondamentali del progetto è che una pull request possa essere utilizzata come fonte di conoscenza sul comportamento richiesto al sistema. Il lavoro di **Gousios, Pinzger e van Deursen (2014)** fornisce una delle prime caratterizzazioni empiriche su larga scala del modello di sviluppo pull-based. Analizzando 166.884 pull request provenienti da 291 progetti, gli autori mostrano che la PR non rappresenta soltanto un contenitore della modifica implementata, ma un punto di aggregazione di codice, discussioni, decisioni, metadati e relazioni con altri artefatti. Tra le ragioni di chiusura senza merge emergono inoltre casi di modifiche duplicate, superate, in conflitto o ritenute non necessarie. Queste categorie sono particolarmente rilevanti per PR-to-Requirements, poiché anticipano il problema di stabilire se un requisito ricostruito sia nuovo oppure già rappresentato, modificato o contraddetto da conoscenza precedente.

Un antecedente ancora più diretto è **Viviani et al. (2021)**, che studiano la presenza di informazioni progettuali nelle discussioni delle pull request. Il lavoro segmenta le discussioni di 34 PR di Node.js, Rails e Rust in 10.790 paragrafi e addestra classificatori per riconoscere quelli contenenti informazioni di design. I risultati mostrano che una quota di conoscenza progettuale rimane nelle conversazioni degli sviluppatori senza essere trasferita nella documentazione ufficiale. Il contributo è importante perché dimostra che la PR può essere trattata come una sorgente strutturabile di conoscenza e non soltanto come evidenza dell'implementazione finale.

Per PR-to-Requirements, questi risultati giustificano l'assunzione di fondo del progetto: **una PR può contenere evidenze sufficienti per ricostruire almeno parte dell'intento funzionale che ha motivato una modifica**. Al tempo stesso, mostrano che tale informazione è spesso dispersa, implicita e dipendente dal contesto del progetto.

2.2 Separare bisogno, chiarimento e soluzione

Ricostruire un requisito a partire da un artefatto di sviluppo è diverso dal riassumere il testo della PR. Un rischio centrale è trasformare una scelta implementativa in un requisito oppure confondere il problema con la soluzione adottata.

Merten et al. (2016) affrontano un problema affine negli issue tracker, distinguendo tre categorie informative: *request*, *clarification* e *solution proposal*. Su 599 issue, 3.519 commenti e oltre 11.000 frasi, il lavoro mostra che la richiesta funzionale vera e propria può essere mescolata a chiarimenti e proposte tecniche. L'identificazione diventa inoltre più difficile quando si scende dalla granularità dell'intero issue alla singola frase.

Questa distinzione è direttamente trasferibile al progetto. Un requisito ricostruito dovrebbe rappresentare **ciò che il sistema deve fare**, mentre dettagli quali

classi, API, algoritmi, file modificati o strategie implementative dovrebbero essere mantenuti separati, a meno che costituiscano essi stessi un vincolo richiesto e supportato dall'evidenza. Il Requirement Assessment Agent dovrebbe quindi verificare non soltanto la qualità linguistica, ma anche il corretto livello di astrazione.

Il lavoro di **Antoniol et al. (2008)** rafforza questo punto mostrando che il testo delle change request contiene segnali utili per distinguere bug ed enhancement. Per PR-to-Requirements ciò suggerisce che l'intento della modifica — ad esempio nuova funzionalità, correzione di comportamento, refactoring o intervento puramente tecnico — può influenzare il modo in cui il requisito deve essere ricostruito. Non ogni PR, infatti, implica necessariamente un nuovo requisito funzionale.

2.3 Evidenze eterogenee, traceability e qualità dei dati

La letteratura sul software traceability mostra che relazioni importanti tra artefatti possono non essere esplicitamente dichiarate. **Rostami Mazrae, Izadi e Heydarnoori (2021)** propongono *Hybrid-Linker*, che combina evidenze testuali e non testuali per recuperare link tra issue e commit. Oltre ai risultati del metodo, il lavoro è metodologicamente rilevante perché valuta separatamente il contributo dei componenti e della loro combinazione. Questo principio di **ablation** è direttamente applicabile a PR-to-Requirements: per dimostrare l'utilità di un evaluator o della memoria non è sufficiente mostrare il funzionamento del sistema completo, ma occorre confrontare configurazioni nelle quali tali componenti siano attivati o disattivati in modo controllato.

Anche **Guo, Cheng e Cleland-Huang (2017)** è rilevante per la componente di memoria. Il loro approccio di traceability semantica mostra l'utilità di rappresentazioni in grado di recuperare artefatti semanticamente collegati anche quando non condividono lo stesso lessico. Nel nostro sistema, lo stesso principio può essere impiegato per recuperare dalla memoria persistente i requisiti storici più vicini al nuovo requisito candidato, da sottoporre poi all'Evaluator per distinguere tra requisito nuovo, duplicato, sovrapposto, raffinamento o conflitto.

Infine, **Bachmann et al. (2010)** mostrano che i collegamenti espliciti tra bug e commit possono essere incompleti e che dataset costruiti automaticamente possono contenere falsi negativi e bias. Il messaggio metodologico è essenziale: l'assenza di un'informazione nella PR non implica necessariamente che tale informazione non esista nel processo di sviluppo. Di conseguenza, la valutazione di PR-to-Requirements deve distinguere tra **errore del modello** e **incompletezza dell'input**, e non dovrebbe utilizzare il solo giudizio di un LLM come ground truth finale.

Nel complesso, questa prima linea di ricerca porta a tre conclusioni: le pull request contengono conoscenza utile ma rumorosa; la ricostruzione deve separare bisogno e soluzione; la valutazione deve rimanere collegata alle evidenze originarie e tenere conto dell'incompletezza dei repository.

3. Large Language Models nel Requirements Engineering

3.1 Dalla generazione alla qualità del requisito

L'applicazione degli LLM al Requirements Engineering ha progressivamente spostato l'attenzione dalla semplice generazione di testo alla qualità, revisione e coerenza degli artefatti prodotti.

Ronanki, Berger e Horkoff (2023) studiano ChatGPT come supporto all'elicitation confrontandone le risposte con quelle di esperti. I risultati indicano buone prestazioni su aspetti quali atomicità, consistenza, correttezza e comprensibilità, ma maggiori difficoltà su ambiguità e fattibilità. Il lavoro rappresenta una baseline concettuale importante per PR-to-Requirements: una singola generazione LLM può produrre output linguisticamente plausibili, ma ciò non garantisce che il requisito sia semanticamente corretto e adeguatamente fondato sul contesto.

Più vicino al nostro task è **Quattrocchi et al. (2026)**, che studiano sia la generazione di user story sia la capacità degli LLM di valutarne la qualità. Gli autori utilizzano dieci LLM e costruiscono un ampio corpus di user story, confrontando output di modelli, studenti ed esperti. Per la valutazione semantica definiscono un codebook con criteri espliciti — tra cui specificità della feature, chiarezza della motivazione, orientamento al problema, chiarezza linguistica e consistenza interna — e mostrano che l'accordo tra LLM e annotatori umani cresce quando il modello riceve istruzioni operative e esempi dettagliati. Questo risultato è particolarmente importante per il Requirement Assessment Agent: il valutatore non dovrebbe ricevere una richiesta generica del tipo “controlla se il requisito è buono”, ma una **rubrica esplicita, versionata e verificabile**.

Lubos et al. (2024) affrontano direttamente la quality assurance dei requisiti con riferimento alle caratteristiche di qualità di ISO 29148. Il modello deve classificare un requisito, motivare la decisione e proporre una riscrittura. Le caratteristiche considerate includono appropriatezza, completezza, conformità, correttezza, fattibilità, necessità, singolarità, non ambiguità e verificabilità. Lo studio mostra che spiegazioni e riscritture possono essere utili, ma non uniformemente affidabili e con variabilità tra domini. Per PR-to-Requirements questo lavoro fornisce una base concreta per strutturare il feedback dell'Evaluator in termini di punteggi, difetti individuati, evidenze e istruzioni di revisione.

3.2 Completezza, inconsistenza e grounding

La qualità di un requisito non è soltanto locale. Quando i requisiti vengono accumulati nel tempo, diventano rilevanti anche le relazioni tra elementi del corpus.

Fantechi et al. (2023) valutano ChatGPT nell'identificazione di inconsistenze tra requisiti in linguaggio naturale. Il modello riesce a individuare alcuni conflitti

non banali, ma produce anche omissioni e falsi allarmi. Il risultato suggerisce di utilizzare l’LLM come componente di supporto, non come oracolo. Nel nostro caso, l’Evaluator può ricevere un piccolo insieme di requisiti storici semanticamente rilevanti e classificare la relazione con il candidato, ma tale decisione deve essere tracciabile e valutata rispetto a un gold set quando disponibile.

Preda et al. (2024) studiano invece la revisione della copertura tra requisiti di alto e basso livello. Il principio è utile anche nel nostro scenario: la valutazione può controllare se il requisito ricostruito copra gli aspetti rilevanti dell’evidenza disponibile e, simmetricamente, se il corpus storico contenga già requisiti equivalenti o parzialmente sovrapposti.

Rodriguez, Dearstyne e Cleland-Huang (2023) mostrano inoltre che il risultato dei task di traceability basati su LLM è sensibile alla formulazione del prompt e al modo in cui viene presentato il contesto. Per un esperimento rigoroso, template, modello, parametri e strategia di retrieval devono quindi essere congelati e versionati.

Questa letteratura porta a un punto centrale: **un LLM può essere utile sia come generatore sia come valutatore, ma il valore del valutatore dipende fortemente dalla rubrica, dal contesto e dall’evidenza che gli vengono forniti.**

4. Sistemi multi-agente e pattern generator–critic

4.1 Multi-agent nel Requirements Engineering

MARE — Multi-Agents Collaboration Framework for Requirements Engineering di **Jin et al. (2024)** è uno dei precedenti architetturali più vicini. Il framework scompone il processo di Requirements Engineering in elicitation, modeling, verification e specification, assegnando ruoli distinti a Stakeholders, Collector, Modeler, Checker e Documenter. Gli artefatti intermedi vengono condivisi in un workspace e, quando il Checker rileva errori, le fasi precedenti vengono rieseguite. Il lavoro include inoltre un confronto con baseline individuali.

MARE supporta due idee centrali di PR-to-Requirements: la **separazione delle responsabilità** tra produzione e controllo e la presenza di un **ciclo di revisione**. La differenza principale è che il workspace di MARE costituisce soprattutto memoria di lavoro della singola esecuzione, mentre PR-to-Requirements richiede una memoria persistente e cross-PR dei requisiti già validati.

SimAC di **Li et al. (2024)** è un ulteriore riferimento peer-reviewed. Il framework genera acceptance criteria simulando la collaborazione di ruoli agili secondo una sequenza *create-update-update*. I risultati riportati mostrano miglioramenti di completezza e validità rispetto a prompting più semplice. Sebbene il task parta da user story già disponibili, SimAC rafforza l’ipotesi che ruoli differenti

possano contribuire in modo complementare alla qualità di un artefatto di requisito.

4.2 Revisione iterativa: Self-Refine, Cross-Refine e limiti dell’auto-correzione

La letteratura generale sugli agenti LLM mostra però che **“più agenti” non equivale automaticamente a “migliore qualità”**. È più utile ragionare sul tipo di informazione che il passaggio di revisione aggiunge.

Self-Refine di **Madaan et al. (2023)** mostra che lo stesso modello può generare, criticare e riscrivere iterativamente un output ottenendo miglioramenti su diversi task. Tuttavia, questa configurazione non dimostra il valore di due agenti distinti: il beneficio può derivare semplicemente da una seconda opportunità di elaborazione.

Cross-Refine di **Wang et al. (2025)** è metodologicamente più vicino al progetto: un generator produce l’output, un critic distinto fornisce feedback e suggerimenti concreti, e il generator utilizza tali indicazioni per riscrivere. Nel setting studiato, la separazione generator–critic supera Self-Refine e le ablation mostrano che sia la critica sia i suggerimenti specifici contribuiscono al miglioramento. Per PR-to-Requirements questo suggerisce che l’Assessment Agent debba restituire **feedback strutturato e azionabile**, ad esempio: affermazioni non supportate, informazioni mancanti, problemi di atomicità, duplicati candidati, conflitti e istruzioni precise per la revisione.

La contro-evidenza di **Huang et al. (2024)** è altrettanto importante: chiedere a un LLM di “ricontrollarsi” senza nuova informazione affidabile può non migliorare il risultato e, in alcuni casi, peggiorarlo. Ne deriva un principio di progettazione forte: l’Evaluator deve possedere un **vantaggio informativo** rispetto al primo passaggio del Generator, attraverso una rubrica esplicita, evidenze della PR e — quando la memoria è attiva — requisiti storici recuperati dal sistema.

Reflexion di **Shinn et al. (2023)** aggiunge la dimensione temporale: il feedback viene trasformato in memoria episodica riutilizzabile nei tentativi successivi senza modificare i pesi del modello. Nel nostro progetto la memoria non dovrebbe contenere semplicemente “riflessioni” dell’LLM, ma oggetti verificabili: requisito, provenienza, PR di origine, stato di validazione, embedding, relazioni di duplicazione/conflitto e versione.

4.3 Il contributo dei lavori di Benedetta Donato

La linea di ricerca di **Benedetta Donato** è particolarmente rilevante perché offre precedenti vicini sia al loop generator–evaluator sia al confronto empirico tra architetture single-LLM e multi-agent.

In **MultiMind: A Plug-in for the Implementation of Development Tasks Aided by AI Assistants** (**Donato et al., 2025**), il caso d’uso sulla generazione di documentazione implementa un pattern molto simile a quello

previsto in PR-to-Requirements: un assistente genera il contenuto, un secondo assistente ne valuta la qualità e, se il giudizio è insufficiente, il task di generazione viene rieseguito fino all’approvazione o al raggiungimento di un limite di iterazioni. La differenza è nel dominio: MultiMind genera documentazione di codice e non ricostruisce requisiti, inoltre non utilizza una memoria persistente del corpus tramite MCP. Proprio per questo costituisce un precedente diretto del **meccanismo di quality loop**, mentre il nostro lavoro lo trasferisce al Requirements Engineering e aggiunge la dimensione della consistenza storica.

In **An Evaluation of Role-Based Multi-Agent Code Generation on Repository-Scale Problems** (Donato et al., 2026), gli autori confrontano una soluzione LLM-only, una pipeline multi-agent sequenziale e una pipeline multi-agent riflessiva su 12 repository Java reali. Il task procede nella direzione opposta rispetto alla nostra tesi — da requisiti/scenari verso il codice — ma il disegno sperimentale è fortemente trasferibile. Il lavoro mostra l’importanza di confrontare la soluzione agentica con una baseline single-LLM sullo stesso campione e con metriche comuni, evitando di attribuire automaticamente i miglioramenti alla sola presenza di più agenti.

Infine, **Studying How Configurations Impact Code Generation in LLMs: The Case of ChatGPT** (Donato et al., 2025) evidenzia la variabilità degli output LLM al variare della configurazione e tra repliche della stessa richiesta. Per la nostra valutazione ciò implica che ciascuna configurazione sperimentale dovrebbe essere eseguita più volte, mantenendo fissi modello, versione, temperatura, top-p, prompt e pipeline.

Un collegamento ancora più diretto con il nostro stage è fornito da **PR4Code: A Pull Requests Dataset for AI Code Generation** (Donato et al., 2026), pubblicato su *IEEE Access*. PR4Code adotta la pull request come unità primaria del task e raccoglie **13.339 PR reali**, di cui 4.508 Java e 8.831 Python, associate a **46.334 commit** provenienti da **1.055 repository GitHub**. Il dataset conserva titolo e body della PR, storia dei commit, metadati e modifiche cumulative al codice. Il contatto con PR-to-Requirements è particolarmente forte perché il paper utilizza esplicitamente **titolo e body come descrizione in linguaggio naturale del task**: nel loro scenario tale descrizione viene usata per generare la modifica software, mentre nel nostro viene usata per ricostruire il requisito funzionale che la modifica esprime. PR4Code fornisce quindi non solo un dataset di partenza citabile e riproducibile, ma anche il ponte empirico tra le due direzioni del problema, **descrizione della PR → codice** e **descrizione della PR → requisito**.

Questi lavori consentono quindi di collocare PR-to-Requirements in continuità con una linea di ricerca sulla collaborazione tra AI assistant e sulla valutazione repository-scale. Il contributo originale non consiste più soltanto nell’invertire la direzione requisiti→codice: grazie a PR4Code, il progetto può partire dallo **stesso artefatto sperimentale reale** studiato dalla tutor e aggiungere ciò che il dataset non contiene, cioè la ricostruzione esplicita del requisito, il quality loop Generator-Evaluator e una memoria persistente per la coerenza del corpus.

5. Memoria persistente, retrieval e Model Context Protocol

5.1 La memoria come componente del sistema

Una memoria dei requisiti storici può supportare un tipo di controllo che non è disponibile osservando una PR in isolamento. Un nuovo requisito può essere linguisticamente corretto e fedele alla PR, ma al tempo stesso duplicare un requisito già presente, raffinarlo, sostituirlo o entrare in conflitto con esso.

L'idea trova antecedenti nella letteratura sulla traceability e sull'uso della storia del progetto. **Tsuchiya et al. (2015)** utilizzano il configuration management log per recuperare collegamenti tra requisiti e codice e per distinguere elementi comuni da elementi specifici. In modo analogo, la memoria di PR-to-Requirements deve essere considerata non come un semplice archivio, ma come una **fonte attiva di contesto storico**.

Dal punto di vista del retrieval, il lavoro di **Guo et al. (2017)** suggerisce l'utilità di ricerca semantica per recuperare artefatti concettualmente affini anche quando il lessico differisce. Un'implementazione naturale consiste quindi in uno store persistente affiancato da un indice semantico o ibrido, dal quale selezionare un numero limitato di candidati da fornire all'Evaluator.

5.2 MCP non è la memoria

Nel progetto, **Model Context Protocol (MCP)** deve essere descritto correttamente come **protocollo di accesso** alla memoria e non come memoria stessa. Il database, il vector store e le politiche di persistenza risiedono dietro il server MCP; MCP standardizza invece il modo in cui gli agenti scoprono e invocano operazioni su tali risorse.

Nella specifica MCP 2026-07-28, il protocollo utilizza JSON-RPC 2.0 e supporta capability come *Resources*, *Tools* e *Prompts*. Per PR-to-Requirements le operazioni più naturali sono tool applicativi limitati e riproducibili, ad esempio:

```
search_requirements(query, project_id, top_k)
get_requirement(requirement_id)
find_candidate_relations(candidate_text, project_id)
```

Le operazioni con side effect, come la persistenza di un requisito, dovrebbero essere riservate a un controller dopo l'accettazione. In questo modo il Requirement Assessment Agent può interrogare la conoscenza storica in modalità read-only, mentre un output non ancora validato non può contaminare la memoria.

Questa separazione offre anche un vantaggio scientifico. È possibile distinguere almeno tre componenti: **store persistente**, **strategia di retrieval** e **protocollo MCP**. L'eventuale miglioramento della qualità non deve essere attribuito

genericamente a MCP: dipende dal contenuto memorizzato, dalla qualità del retrieval e dal modo in cui i risultati vengono utilizzati dagli agenti.

5.3 Memoria temporale e rischio di leakage

La memoria introduce un problema metodologico ulteriore: la conoscenza deve rispettare l'ordine temporale. Se si processano PR storiche, la PR al tempo t non deve avere accesso a requisiti derivati da PR future. Una valutazione corretta deve quindi costruire la memoria cronologicamente o utilizzare snapshot controllati.

È inoltre utile distinguere due scenari sperimentali:

1. **controlled memory**, nella quale la memoria contiene requisiti precedenti già validati e serve a misurare il beneficio puro del retrieval;
2. **online memory**, nella quale il sistema salva progressivamente i propri output e può quindi propagare errori nel tempo.

Il primo scenario è più adatto all'esperimento principale; il secondo è utile come analisi successiva del comportamento realistico del sistema.

6. Dataset sperimentale e costruzione del gold standard

Nel nostro studio è necessario distinguere due elementi che non coincidono: il **dataset di pull request usato come sorgente sperimentale** e il **gold standard dei requisiti funzionali** con cui valutare gli output. Il paper PR4Code chiarisce ora in modo preciso il primo elemento; il secondo deve essere costruito appositamente per PR-to-Requirements.

6.1 PR4Code come dataset di partenza

PR4Code: A Pull Requests Dataset for AI Code Generation, di Benedetta Donato, Leonardo Mariani, Daniela Micucci e Oliviero Riganelli, è stato pubblicato su *IEEE Access* nel 2026 (DOI [10.1109/ACCESS.2026.3713096](https://doi.org/10.1109/ACCESS.2026.3713096)). Il dataset contiene **13.339 pull request reali**, di cui 4.508 Java e 8.831 Python, associate a **46.334 commit** e provenienti da **1.055 repository GitHub**. Gli autori rendono pubblici anche dataset e script di costruzione tramite OSF, rendendo la raccolta riproducibile e aggiornabile.

I repository sono selezionati tra progetti Java e Python con più di 500 GitHub star e con PR successive al 3 ottobre 2023, vincolo introdotto come mitigazione del possibile training-data leakage. Le PR considerate sono chiuse e devono rappresentare attività come feature development, bug fixing o miglioramenti incrementali. Inoltre, titolo e body devono contenere almeno una keyword indicativa dell'attività, avere una lunghezza complessiva di almeno **100 caratteri** e provenire da branch con nomi semanticamente compatibili con feature, task, fix o categorie analoghe.

Questi criteri sono particolarmente adatti al nostro stage perché selezionano PR con un'intenzione di sviluppo relativamente esplicita. Nel campione casuale usato dagli autori per la validazione esterna, il 19% delle PR possiede un body vuoto e il 27% non raggiunge i 100 caratteri complessivi; PR4Code esclude quindi una parte dei casi per i quali un task basato soltanto sul testo sarebbe scarsamente informativo.

Per ogni PR il dataset conserva titolo e body, metadati, sequenza e messaggi dei commit, file modificati e diff cumulativo tra versione base e finale. Il paper osserva inoltre che **l'intento di alto livello è catturato principalmente a livello di PR**, mentre i commit descrivono azioni più operative e granulari. Questo risultato è direttamente coerente con PR-to-Requirements: titolo e body sono proprio il livello testuale dal quale vogliamo ricostruire il comportamento richiesto.

6.2 Il contatto diretto con PR-to-Requirements

PR4Code nasce per valutare sistemi di code generation, ma il suo workflow è quasi speculare al nostro. Nell'esperimento di applicabilità del paper, una pipeline agentica riceve **titolo e body della PR**, pianifica la modifica, recupera il contesto del repository, genera il codice e infine compila ed esegue i test. Il nostro sistema parte dallo stesso input, ma produce un artefatto differente:

```
PR4Code:          PR title + body -> interpretazione dell'intento -> codice
PR-to-Requirements: PR title + body -> interpretazione dell'intento -> requisito
```

PR4Code costituisce quindi un punto di partenza molto più forte di un generico corpus GitHub: le PR sono curate come attività di sviluppo reali e completate, e la descrizione naturale è già trattata come rappresentazione del task. La nostra tesi introduce il passaggio che manca nel dataset: trasformare quell'intento in una **specifica funzionale normalizzata**, sottoporla a quality assessment e metterla in relazione con requisiti storici.

Il paper segnala però che molte descrizioni non sono completamente autonome: il **55% delle PR Java** e il **52% delle PR Python** contiene almeno un riferimento a un artefatto esterno. Per un esperimento che usa soltanto titolo e body è quindi importante distinguere PR *self-contained* da PR *cross-referenced*. In questo modo un requisito incompleto non viene automaticamente interpretato come errore del modello quando l'informazione necessaria non è presente nell'input.

6.3 PR4Code non è il gold standard dei requisiti

La precedente nota secondo cui PR4Code non fosse pubblicamente documentato deve quindi essere rimossa: il dataset è reale, pubblico e direttamente attribuibile a Donato et al. Allo stesso tempo, **PR4Code non contiene un requisito funzionale gold per ciascuna PR**. Il riferimento del dataset è l'attività di sviluppo effettivamente realizzata, descritta tramite commit, file e

patch. Questo è sufficiente per il task originale di code generation, ma non fornisce una frase-requisito da utilizzare direttamente come target della nostra valutazione.

La conseguenza metodologica è importante: **PR4Code risolve il problema del corpus di partenza, ma il gold standard per requirement reconstruction deve essere aggiunto dalla tesi.** Commit e diff possono aiutare gli annotatori a verificare ex post quale comportamento sia stato effettivamente modificato, ma non devono introdurre nel requisito gold informazioni che non erano ricostruibili da titolo e body. Il codice deve quindi essere usato come evidenza di corroborazione e adjudication, non come fonte nascosta di dettagli da aggiungere al target.

6.4 Protocollo proposto per il gold standard

Per un sottoinsieme di PR4Code, almeno due annotatori dovrebbero produrre indipendentemente un requisito a partire da **titolo e body soltanto**. Per ogni caso andrebbero registrati: testo del requisito, evidenze testuali che lo supportano, tipo generale della modifica, self-containedness, confidenza e una decisione **extractable / not_extractable**. Quest'ultimo campo è necessario perché non tutte le PR implicano un requisito funzionale: refactoring puramente tecnici, aggiornamenti infrastrutturali o descrizioni troppo dipendenti da artefatti esterni non dovrebbero costringere gli annotatori a inventare un comportamento utente.

Dopo l'annotazione indipendente, commit e diff possono essere consultati nella fase di verifica e **adjudication**. Se l'implementazione contiene dettagli non presenti nel testo, tali elementi non devono essere promossi automaticamente a requisito gold; il caso deve invece essere marcato come parzialmente osservabile o non ricostruibile dal solo input. Il processo deve essere supportato da un codebook e da una misura dell'accordo tra annotatori, mantenendo separato il Requirement Assessment Agent dalla valutazione finale.

Per valutare anche la memoria persistente serve un secondo livello di gold standard. Su un campione di requisiti semanticamente vicini è utile annotare relazioni come **NEW, DUPLICATE, OVERLAPS, REFINES, SUPERSEDES** e **CONFLICTS**. Questo permette di misurare precision, recall e F1 del retrieval e della decisione dell'Evaluator, invece di limitarsi a una valutazione qualitativa della memoria.

6.5 Split, cronologia e limiti

La presenza di 1.055 repository rende possibile uno split **per repository**, preferibile a un random split di PR dello stesso progetto perché riduce il rischio di leakage dovuto a template, naming convention e modifiche quasi duplicate. Quando la memoria è attiva, le PR devono inoltre essere elaborate in ordine cronologico: la PR al tempo t può consultare soltanto requisiti validati derivati da PR precedenti.

PR4Code presenta infine alcuni bias dichiarati dagli stessi autori. La soglia di popolarità privilegia repository maturi; il minimo di 100 caratteri e i filtri per keyword e branch favoriscono PR relativamente descrittive; Java e Python non rappresentano tutti gli ecosistemi; e il vincolo temporale riduce ma non elimina la possibilità che modelli più recenti abbiano visto parte del corpus durante il training. Questi limiti non rendono il dataset inadatto: lo rendono un **benchmark iniziale controllabile e ricco di segnale testuale**, ma impongono cautela nella generalizzazione a PR più brevi, rumorose o provenienti da progetti meno strutturati.

Il punto metodologico da conservare nella tesi è quindi netto: PR4Code è il dataset di partenza; il gold standard dei requisiti è un nuovo layer di annotazione costruito sopra PR4Code da PR-to-Requirements. Questa distinzione valorizza direttamente il lavoro di Donato et al. e, allo stesso tempo, identifica con precisione uno degli artefatti originali prodotti dalla tesi.

7. Sintesi comparativa della letteratura

Lavoro	Artefatto di partenza	Output / task	Revisione LLM	Multi-agent	Memoria persistente	Rilevanza per PR-to-Requirements
Gousios et al. (2014)	Pull request	Studio empirico PR	No	No	No	Fondamento socio-tecnico e dataset
Viviani et al. (2021)	Discussioni PR	Design information localization	No	No	No	Evidenza che le PR contengono conoscenza progettuale
Merten et al. (2016)	Issue/comment	Feature request detection	No	No	No	Separazione request / clarification / solution

Lavoro	Artefatto di partenza	Output / task	Revisione LLM	Multi-agent	Memoria persistente	Rilevanza per PR-to-Requirements
Rostami Mazrae et al. (2021)	Issue + commit + metadati	Trace link recovery	No	No	No	Combinazione di componenti e ablation
Guo et al. (2017)	Artefatti software	Semantic trace-ability	No	No	No	Fondamento per retrieval semantico
Ronanki et al. (2023)	Domande di elicitation	Requisiti/risposte	No	No	No	Baseline single-LLM
Lubos et al. (2024)	Requisito	Quality assessment + rewrite	Sì	No	No	Rubrica dell'Evaluator
Quattrocchi et al. (2026)	Elicitation simulata / user story	Generazione + assessment	Valutazione	Ruoli LLM	No	Generatore e LLM evaluator
SimAC (Li et al., 2024)	User story	Acceptance criteria	Sì	Sì	No	Revisione sequenziale role-based
MARE (Jin et al., 2024)	Input RE	Elicitation-SSRS	Sì	Sì	Solo workspace	Precedente multi-agent nel RE
Self-Refine (Madaan et al., 2023)	Task NLP	Iterative refinement	Sì	No	No	Baseline di self-revision
Cross-Refine (Wang et al., 2025)	Task NLP	Generator-Sritic	Sì	revisione / ruoli distinti	No	Pattern più vicino al loop

Lavoro	Artefatto di partenza	Output / task	Revisione LLM	Multi-agent	Memoria persistente	Rilevanza per PR-to-Requirements
Reflexion (Shinn et al., 2023)	Task agentici	Reflection + retry	Sì	No/agentico	Episodica	Fondamento concettuale della memoria
MultiMind (Donato et al., 2025)	Task di sviluppo	Generazione + quality loop	Sì	Multi-assistant	No	Precedente diretto del loop della tutor
Donato et al. (2026)	Requisiti/scheda	Repository-scale code generation	Sì	Sì	No	Precedente sperimentale single vs multi-agent
PR4Code (Donato et al., 2026)	Titolo/body PR + artefatti repository	Dataset e benchmark per feature-level code generation	No loop dichiarato	Sì, nella demo agentica	No	Dataset di partenza diretto del nostro studio
PR-to-Requirements	Titolo + body PR	Requisito funzionale	Sì	Sì	Sì, via MCP	Combinazione non coperta dai lavori precedenti

8. Research gap e posizionamento del progetto

La letteratura analizzata copre separatamente quasi tutte le componenti necessarie al progetto:

- le pull request contengono informazione progettuale e contestuale utile;

- esistono metodi per riconoscere richieste funzionali e distinguere bisogno e soluzione;
- gli LLM possono generare artefatti di requisito e valutarne alcuni attributi di qualità;
- i loop di revisione possono migliorare l’output quando il feedback è informativo;
- un critic distinto può essere più utile della sola auto-critica;
- la storia del progetto e il retrieval semantico possono supportare la relazione tra artefatti;
- architetture multi-agent sono state studiate sia nel Requirements Engineering sia nella code generation a scala repository;
- **PR4Code mette a disposizione un dataset pubblico, ampio e riproducibile di PR reali nel quale titolo e body sono già trattati come descrizione naturale del task di sviluppo;**
- MCP fornisce un’interfaccia standardizzata per collegare agenti a dati e tool persistenti.

La disponibilità di PR4Code elimina quindi un’incertezza presente nella prima bozza: **il problema non è più trovare un corpus citabile di pull request**. Il gap si sposta su ciò che PR4Code non fornisce, cioè requisiti funzionali gold ricostruiti dalle descrizioni, un protocollo di quality assessment orientato al Requirements Engineering e relazioni storiche tra requisiti. Ciò che **non emerge come combinazione consolidata nei lavori analizzati** è una pipeline che utilizzi PR reali di questo tipo per ricostruire un requisito funzionale mediante un generator LLM, sottoponga tale requisito a un critic evidence-grounded, utilizzi una memoria persistente dei requisiti storici per identificare duplicazioni e incoerenze e separi sperimentalmente l’effetto del critic da quello della memoria.

Il contributo scientifico di PR-to-Requirements può quindi essere formulato non come la semplice costruzione di “un sistema con due agenti”, ma come la valutazione controllata di **quanto e in quali condizioni**:

1. un **Requirement Assessment Agent** migliori la qualità locale del requisito rispetto a una singola generazione;
2. una **memoria persistente dei requisiti**, interrogata tramite MCP, migliori la coerenza globale del corpus e il riconoscimento di duplicazioni o conflitti;
3. la combinazione di valutatore e memoria produca un beneficio aggiuntivo rispetto ai due componenti considerati separatamente.

Una formulazione sperimentale naturale è quindi un disegno fattoriale 2×2 :

Configurazione	Evaluator	Memoria	Scopo
A — Baseline	No	No	Single-shot LLM
B — Loop	Sì	No	Effetto del critic
C — Memory	No	Sì	Effetto della memoria
D — Full	Sì	Sì	Effetto combinato e interazione

Per evitare confondenti, modello, versione, input, prompt di base e formato del requisito dovrebbero rimanere costanti tra le configurazioni. La stessa PR dovrebbe essere processata in più repliche per controllare il non-determinismo. Oltre alla qualità del requisito, la valutazione dovrebbe includere costo computazionale, token, numero di chiamate, latenza e stabilità.

9. Implicazioni per la progettazione del Requirement Assessment Agent

Dalla letteratura emergono criteri concreti che possono essere utilizzati per definire il comportamento dell'Evaluator. Un output strutturato potrebbe includere:

```
{
  "scores": {
    "fidelity": 0,
    "completeness": 0,
    "atomicity": 0,
    "clarity": 0,
    "verifiability": 0,
    "abstraction": 0
  },
  "unsupported_claims": [],
  "missing_information": [],
  "duplicate_candidates": [],
  "conflicts": [],
  "revision_instructions": [],
  "accept": false
}
```

In particolare, il valutatore dovrebbe controllare che:

- il requisito sia supportato da titolo e body della PR;
- non vengano introdotti vincoli o comportamenti non presenti nell'evidenza;
- il requisito descriva il comportamento desiderato e non semplicemente la soluzione implementativa;
- il testo sia atomico, chiaro, verificabile e sufficientemente completo;
- gli eventuali requisiti recuperati dalla memoria siano usati per identificare duplicati, sovrapposizioni, raffinamenti o conflitti;
- il feedback fornito al Generator sia specifico e azionabile.

La decisione finale usata per la valutazione scientifica, tuttavia, non dovrebbe essere affidata allo stesso LLM che costituisce l'Assessment Agent: è preferibile mantenere un **protocollo di valutazione esterno**, con annotatori umani e/o un giudice indipendente, per evitare una valutazione circolare.

10. Conclusion

Lo stato dell'arte mostra che PR-to-Requirements si colloca all'intersezione di quattro aree mature ma finora solo parzialmente integrate: mining delle pull request e software repositories, LLM per Requirements Engineering, architetture generator-critic/multi-agent e memoria esterna con retrieval.

I lavori sulle pull request dimostrano che tali artefatti contengono conoscenza progettuale e funzionale, ma anche rumore, ambiguità e informazioni incomplete. Gli studi LLM4RE indicano che i modelli sono in grado di generare e valutare requisiti, purché la qualità sia definita attraverso rubriche esplicite e il risultato sia verificato. La letteratura multi-agent mostra che feedback e revisione possono migliorare una generazione, ma non supporta una regola generale secondo cui aumentare il numero di agenti migliori automaticamente l'output: il critic deve aggiungere informazione utile e indipendente. Infine, la memoria persistente rende possibile passare dalla qualità del singolo requisito alla coerenza del corpus nel tempo; MCP offre un boundary standardizzato e auditabile per consentire agli agenti di accedere a tale memoria, senza coincidere con la memoria stessa.

La disponibilità di PR4Code rafforza ulteriormente questo posizionamento: il progetto può essere valutato su un corpus pubblico e riproducibile che già rappresenta titolo e body delle PR come descrizione del task, mentre la tesi aggiunge il nuovo layer di annotazione necessario a trasformare tali descrizioni in requisiti gold. Il gap risultante è quindi sufficientemente specifico e sperimentalmente verificabile: **ricostruire requisiti funzionali da pull request tramite un loop Generator-Evaluator evidence-grounded, integrarlo con una memoria persistente cross-PR accessibile via MCP e misurare separatamente il contributo di revisione, memoria e loro interazione.**

Riferimenti

1. G. Gousios, M. Pinzger, A. van Deursen, “An Exploratory Study of the Pull-based Software Development Model,” *ICSE*, 2014, pp. 345–355. DOI: 10.1145/2568225.2568260.
2. G. Viviani, M. Famelis, X. Xia, C. Janik-Jones, G. C. Murphy, “Locating Latent Design Information in Developer Discussions: A Study on Pull Requests,” *IEEE Transactions on Software Engineering*, 47(7), 2021, pp. 1402–1413. DOI: 10.1109/TSE.2019.2924006.
3. T. Merten, M. Falis, P. Hübner, T. Quirchmayr, S. Bürsner, B. Paech, “Software Feature Request Detection in Issue Tracking Systems,” *IEEE International Requirements Engineering Conference*, 2016, pp. 166–175. DOI: 10.1109/RE.2016.8.

4. P. Rostami Mazrae, M. Izadi, A. Heydarnoori, “Automated Recovery of Issue-Commit Links Leveraging Both Textual and Non-textual Data,” *ICSME*, 2021, pp. 263–273. DOI: 10.1109/ICSME52107.2021.00030.
5. J. Guo, J. Cheng, J. Cleland-Huang, “Semantically Enhanced Software Traceability Using Deep Learning Techniques,” *ICSE*, 2017, pp. 3–14. DOI: 10.1109/ICSE.2017.9.
6. A. Bachmann, C. Bird, F. Rahman, P. Devanbu, A. Bernstein, “The Missing Links: Bugs and Bug-fix Commits,” *ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 2010, pp. 97–106. DOI: 10.1145/1882291.1882308.
7. G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, Y.-G. Guéhéneuc, “Is It a Bug or an Enhancement? A Text-Based Approach to Classify Change Requests,” *CASCON*, 2008, pp. 304–318. DOI: 10.1145/1463788.1463819.
8. R. Tsuchiya, H. Washizaki, Y. Fukazawa, T. Kato, M. Kawakami, K. Yoshimura, “Recovering Traceability Links between Requirements and Source Code Using the Configuration Management Log,” *IEICE Transactions on Information and Systems*, E98-D(4), 2015, pp. 852–862. DOI: 10.1587/transinf.2014EDP7199.
9. D. Jin, Z. Jin, X. Chen, C. Wang, “MARE: Multi-Agents Collaboration Framework for Requirements Engineering,” arXiv:2405.03256, 2024. DOI: 10.48550/arXiv.2405.03256.
10. G. Quattrocchi, L. Pasquale, P. Spoletini, L. Baresi, “Can LLMs Generate User Stories and Assess Their Quality?,” *IEEE Transactions on Software Engineering*, 52(5), 2026, pp. 1773–1790. DOI: 10.1109/TSE.2026.3670612.
11. Y. Li, J. Keung, Z. Yang, X. Ma, J. Zhang, S. Liu, “SimAC: Simulating Agile Collaboration to Generate Acceptance Criteria in User Story Elaboration,” *Automated Software Engineering*, 31(2), Article 55, 2024. DOI: 10.1007/s10515-024-00448-7.
12. S. Lubos, A. Felfernig, T. N. T. Tran, D. Garber, M. El Mansi, S. P. Erdeniz, V.-M. Le, “Leveraging LLMs for the Quality Assurance of Software Requirements,” *IEEE International Requirements Engineering Conference*, 2024, pp. 389–397. DOI: 10.1109/RE59067.2024.00046.
13. A. Fantechi, S. Gnesi, L. Passaro, L. Semini, “Inconsistency Detection in Natural Language Requirements Using ChatGPT: A Preliminary Evaluation,” *IEEE International Requirements Engineering Conference*, 2023, pp. 335–340. DOI: 10.1109/RE57278.2023.00045.
14. K. Ronanki, C. Berger, J. Horkoff, “Investigating ChatGPT’s Potential to Assist in Requirements Elicitation Processes,” *49th Euromicro Conference on Software Engineering and Advanced Applications*, 2023, pp. 354–361. DOI: 10.1109/SEAA60479.2023.00061.
15. A. D. Rodriguez, K. R. Dearstyne, J. Cleland-Huang, “Prompts Matter: Insights and Strategies for Prompt Engineering in Automated Software Traceability,” *IEEE Requirements Engineering Conference Workshops*, 2023, pp. 455–464. DOI: 10.1109/REW57809.2023.00087.
16. A.-R. Preda, C. Mayr-Dorn, A. Mashkoo, A. Egyed, “Supporting High-

- Level to Low-Level Requirements Coverage Reviewing with Large Language Models,” *MSR*, 2024, pp. 242–253. DOI: 10.1145/3643991.3644922.
17. Q. Wang, T. Anikina, N. Feldhus, S. Ostermann, S. Möller, V. Schmitt, “Cross-Refine,” *COLING*, 2025, pp. 1150–1167. ACL Anthology record: 2025.coling-main.77.
 18. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *NeurIPS*, 2023. DOI: 10.52202/075280-2019.
 19. N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, “Reflexion,” *NeurIPS*, 2023. DOI: 10.52202/075280-0377.
 20. J. Huang et al., “Large Language Models Cannot Self-Correct Reasoning Yet,” *ICLR*, 2024; arXiv:2310.01798.
 21. B. Donato, L. Mariani, D. Micucci, O. Riganelli, M. Somaschini, “MultiMind: A Plug-in for the Implementation of Development Tasks Aided by AI Assistants,” *FSE Companion ’25*, 2025, pp. 1310–1317. DOI: 10.1145/3696630.3730564.
 22. B. Donato, N. Hagar-Dent, A. Worsnop, L. Mariani, V. Terragni, “An Evaluation of Role-Based Multi-Agent Code Generation on Repository-Scale Problems,” *IEEE Software*, 2026. DOI: 10.1109/MS.2026.3701516.
 23. B. Donato, L. Mariani, D. Micucci, O. Riganelli, “Studying How Configurations Impact Code Generation in LLMs: The Case of ChatGPT,” *ICPC*, 2025, pp. 442–453. DOI: 10.1109/ICPC66645.2025.00055.
 24. B. Donato, “Rethinking Software Development Considering Collaboration with AI Assistants,” *ICSE Companion / Doctoral Symposium*, 2025, pp. 154–156. DOI: 10.1109/ICSE-Companion66252.2025.00045.
 25. B. Donato, L. Mariani, D. Micucci, O. Riganelli, “PR4Code: A Pull Requests Dataset for AI Code Generation,” *IEEE Access*, vol. 14, 2026, pp. 108479–108491. DOI: 10.1109/ACCESS.2026.3713096. Dataset and construction scripts: OSF PR4Code.
 26. Model Context Protocol, “Specification 2026-07-28,” official MCP specification, 2026.